Unidad 2: Uso de estilos

DISEÑO DE INTERFACES WEB - 2DAW

Sonia Mastache Maestre



Índice general

1. Introducción 2
2. Estilos en línea basados en etiquetas y en clases 2
   - 2.1. Estilos basados en línea (Inline Styles) 2
   - 2.2. Estilos basados en etiquetas (Tag Selectors) 2
     - Sintaxis y aplicación 3
     - Características principales 3
   - 2.3. Estilos basados en clases (Class Selectors) 3
     - Sintaxis e implementación 3
     - Múltiples clases 4
   - 2.4. Estilos basados en ID (ID Selectors) 5
     - Sintaxis y uso 5
   - 2.5. Tipos de elementos: Bloque y en línea 5
   - 2.6. Modelo de cajas (Box Model) 7
   - 2.7. Jerarquía y especificidad 8
   - 2.8. Mejores prácticas 9
   - 2.9. Lenguaje CSS 9
3. Crear y vincular hojas de estilo en cascada externa 9
4. Herramientas y test de verificación 13
5. Tecnologías y frameworks 14
   - 5.1. Tecnologías de diseño 14
     - Media Queries 14
     - Flexbox 16
     - Grid Layout 16
   - 5.2. Frameworks CSS más populares 17
6. Preprocesadores de estilos. Variables, mixins y funciones 18

1. Introducción

El uso de estilos en el desarrollo web es fundamental para separar la estructura del contenido (HTML) de la presentación visual (CSS). Esta unidad está dedicada a comprender cómo los estilos permiten transformar páginas web simples en interfaces atractivas, accesibles y profesionales. Aprenderás a aplicar estilos de diferentes formas, a organizar y reutilizar reglas CSS, y a utilizar herramientas modernas que facilitan el trabajo con hojas de estilo.

Además, se abordarán conceptos clave como la especificidad, la cascada y las mejores prácticas para mantener el código limpio y escalable. También conocerás tecnologías y frameworks actuales, así como preprocesadores que potencian el desarrollo de estilos avanzados. Al finalizar la unidad, serás capaz de crear, vincular y optimizar hojas de estilo para cualquier proyecto web.

2. Estilos en línea basados en etiquetas y en clases

Los estilos CSS pueden aplicarse de diferentes formas en un documento HTML. En esta sección exploraremos las tres principales metodologías para aplicar estilos: estilos en línea, estilos basados en etiquetas y estilos basados en clases.

Los estilos en línea se aplican directamente a un elemento HTML utilizando el atributo `style`. Esta es la forma más específica de aplicar CSS y tiene la mayor prioridad en la cascada de estilos.

2.1. Estilos basados en línea (Inline Styles)

```
<elemento style="propiedad: valor; propiedad2: valor2;">
    Contenido del elemento
</elemento>
```

Ejemplo práctico:

```
<h1 style="color: red; font-size: 24px; text-align: center;">
    Título Principal
</h1>
<p style="background-color: #f0f0f0; padding: 10px; border: 1px solid #ccc;">
    Este párrafo tiene estilos aplicados en línea.
</p>
```

Importante: Los estilos en línea deben usarse con moderación y principalmente para casos específicos o pruebas temporales, ya que van en contra de las mejores prácticas de separación de contenido y presentación.

2.2. Estilos basados en etiquetas (Tag Selectors)

Los estilos basados en etiquetas se aplican a todos los elementos de un tipo específico en el documento HTML. Se definen utilizando el nombre de la etiqueta como selector.

Sintaxis y aplicación

```
<style>
etiqueta {
    propiedad: valor;
    propiedad2: valor2;
}
</style>
```

Ejemplos prácticos:

```
<style>
h1 {
    color: navy;
    font-family: Arial, sans-serif;
    text-align: center;
}

p {
    line-height: 1.6;
    margin-bottom: 15px;
    text-align: justify;
}

img {
    max-width: 100%;
    height: auto;
    border-radius: 5px;
}
</style>
```

Características principales

- Aplica estilos a todas las instancias de una etiqueta
- Útil para establecer estilos base o por defecto
- Menor especificidad que los estilos en línea
- Facilita la consistencia visual del documento

2.3. Estilos basados en clases (Class Selectors)

Los estilos basados en clases permiten aplicar estilos específicos a elementos que comparten una clase determinada. Son la forma más flexible y reutilizable de aplicar CSS.

Sintaxis e implementación

```
<style>
.nombre-clase {
    propiedad: valor;
```

```
    propiedad2: valor2;
}
</style>

<elemento class="nombre-clase">Contenido</elemento>
```

Ejemplos prácticos:

```
<style>
.destacado {
    background-color: #ffffcc;
    border-left: 4px solid #ffeb3b;
    padding: 10px;
    margin: 15px 0;
}

.boton {
    background-color: #007bff;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

.boton:hover {
    background-color: #0056b3;
}

.texto-grande {
    font-size: 1.2em;
    font-weight: bold;
}
</style>
```

Aplicación en HTML:

```
<p class="destacado">Este párrafo está destacado</p>
<button class="boton">Enviar</button>
<span class="texto-grande">Texto importante</span>
```

Múltiples clases

Es posible aplicar múltiples clases a un mismo elemento:

```
<p class="destacado texto-grande">
    Párrafo con múltiples clases aplicadas
</p>
```

2.4. Estilos basados en ID (ID Selectors)

El selector de ID permite aplicar estilos a un único elemento identificado de forma única en el documento HTML. Se utiliza el carácter # seguido del nombre del ID.

Sintaxis y uso

```
<style>
#nombre-id {
     propiedad: valor;
     propiedad2: valor2;
}
</style>

<elemento id="nombre-id">Contenido</elemento>
```

Ejemplo práctico:

```
<style>
#aviso {
     background-color: #ffe0e0;
     color: #b71c1c;
     padding: 10px;
     border: 2px solid #b71c1c;
     border-radius: 4px;
     font-weight: bold;
     text-align: center;
}
</style>

<div id="aviso">¡Atención! Este mensaje es importante.</div>
```

Características principales:

- El ID debe ser único en todo el documento.
- Tiene mayor especificidad que las clases y etiquetas.
- Útil para destacar o personalizar elementos concretos.
- No se recomienda para estilos reutilizables.

2.5. Tipos de elementos: Bloque y en línea

En HTML y CSS, los elementos se clasifican principalmente en dos tipos según su comportamiento en el flujo del documento: **elementos de bloque** y **elementos en línea**.

Elementos de bloque

Los elementos de bloque (`block`) ocupan todo el ancho disponible de su contenedor y siempre comienzan en una nueva línea. Son ideales para estructurar la página y crear secciones.

Ejemplos comunes: `<div>`, `<p>`, `<h1>`, `<ul>`, `<li>`, `<form>`

```
<div>Este es un elemento de bloque</div>
<p>Otro bloque de texto</p>
```

Características:

- Ocupan todo el ancho disponible.
- Generan un salto de línea antes y después.
- Se pueden aplicar propiedades de tamaño (`width`, `height`), márgenes y relleno.

Elementos en línea

Los elementos en línea (`inline`) sólo ocupan el espacio necesario para su contenido y no generan saltos de línea. Se utilizan para modificar partes de texto o agrupar pequeños fragmentos.

Ejemplos comunes: `<span>`, `<a>`, `<strong>`, `<em>`, `<img>`

```
<p>Texto con <span style="color: red;">palabra en rojo</span> y <a href="#">enlace</a>
```

Características:

- Sólo ocupan el espacio de su contenido.
- No generan saltos de línea.
- No admiten propiedades de tamaño (`width`, `height`) por defecto.
- Se pueden anidar dentro de elementos de bloque.

Propiedad `display`

La propiedad CSS `display` permite cambiar el tipo de comportamiento de un elemento:

```
.bloque {
    display: block;
}
.enlinea {
    display: inline;
}
.enlinea-bloque {
    display: inline-block;
}
```

Ejemplo:

```
<span style="display: block; background: #e3f2fd;">Ahora soy un bloque</span>
```

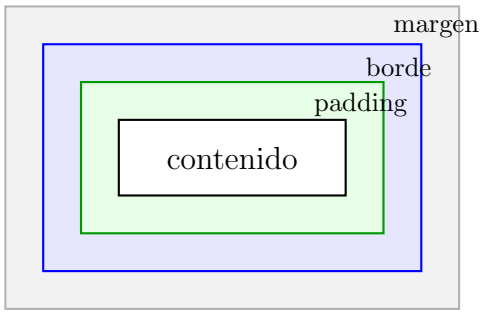
> **Nota:** El valor `inline-block` combina características de ambos tipos: se comporta como en línea pero permite definir tamaño y márgenes.

Lee más: La propiedad display en CSS

2.6. Modelo de cajas (Box Model)

El Modelo de cajas es el principio fundamental que define cómo se renderizan y ocupan espacio los elementos en CSS. Cada elemento se representa como una caja compuesta por:

- **Contenido (content):** El área donde se muestra el texto o imagen.
- **Relleno (padding):** Espacio entre el contenido y el borde.
- **Borde (border):** El marco que rodea el padding y el contenido.
- **Margen (margin):** Espacio exterior entre el borde y otros elementos.



Ejemplo:

```
.caja {
        width: 200px;
        padding: 20px;
        border: 4px solid #007bff;
        margin: 30px;
        background: #e3f2fd;
}
```

Lee Modelo de cajas

Propiedad box-sizing: Por defecto, el ancho y alto sólo incluyen el área de contenido. Si quieres que incluyan padding y borde, usa:

```
.caja {
        box-sizing: border-box;
}
```

Esto facilita la maquetación y evita desbordamientos inesperados.
Lee Modelo de box-sizing

2.7. Jerarquía y especificidad

La especificidad en CSS determina qué estilos se aplican cuando hay conflictos. El orden de prioridad es:

1. **Estilos en línea** (especificidad: 1000)
2. **IDs** (especificidad: 100)
3. **Clases, atributos, pseudo-clases** (especificidad: 10)
4. **Elementos y pseudo-elementos** (especificidad: 1)

¿Qué es la especificidad?
La especificidad es un sistema de puntuación que utiliza CSS para decidir qué regla se aplica si varias coinciden sobre el mismo elemento. Cada selector suma puntos según su tipo:

- Estilos en línea: 1 punto en la columna de los miles (`1000`)
- Selectores de ID: 1 punto en la columna de las centenas (`100`)
- Clases, atributos y pseudo-clases: 1 punto en la columna de las decenas (`10`)
- Elementos y pseudo-elementos: 1 punto en la columna de las unidades (`1`)

¿Por qué se usan estos números? ¿Qué significa "en binario"?
No es realmente binario, sino una notación de "cuatro columnas" (miles, centenas, decenas, unidades) para comparar fácilmente la importancia de cada selector. Por ejemplo:

- Un selector de ID suma 100 (`#menu` → 0-1-0-0)
- Una clase suma 10 (`.rojo` → 0-0-1-0)
- Un elemento suma 1 (`p` → 0-0-0-1)
- Un estilo en línea suma 1000 (`style="..."` → 1-0-0-0)

Cuando hay conflicto, CSS compara de izquierda a derecha: el número más alto en la columna más significativa gana.

Ejemplo de comparación:

```
p { color: blue; }           /* 0-0-0-1 */
.texto { color: green; }     /* 0-0-1-0 */
#especial { color: red; }    /* 0-1-0-0 */
```

Si un elemento tiene los tres selectores, el color será rojo porque el ID tiene mayor especificidad.

> **Nota:**
> Si dos reglas tienen la misma especificidad, gana la que aparece más abajo en el CSS.

Ejemplo de especificidad:

```
<style>
p { color: blue; }                    /* Especificidad: 1 */
.texto { color: green; }              /* Especificidad: 10 */
#especial { color: red; }             /* Especificidad: 100 */
</style>

<p class="texto" id="especial" style="color: purple;">
    Este texto será púrpura (estilo en línea gana)
</p>
```

2.8. Mejores prácticas

- **Evitar estilos en línea** para el código de producción
- **Usar clases** para estilos reutilizables
- **Aplicar estilos a etiquetas** para estilos base globales
- **Mantener la especificidad baja** para facilitar el mantenimiento
- **Separar presentación de contenido** utilizando archivos CSS externos

> **Nota:**
> Los estilos basados en clases son fundamentales para crear interfaces web mantenibles y escalables. Permiten la reutilización de código y facilitan la implementación de sistemas de diseño coherentes.

2.9. Lenguaje CSS

Repasa y aprende
W3Schools CSS
MDN Web Docs
Tabla resumen CSS
Lenguaje CSS
CSS nivel intermedio

3. Crear y vincular hojas de estilo en cascada externa

Las hojas de estilo en cascada externas (CSS externo) son la forma más profesional y escalable de aplicar estilos a una web. Permiten separar completamente el contenido (HTML) de la presentación (CSS), facilitando el mantenimiento, la reutilización y la colaboración en proyectos grandes.

¿Cómo crear una hoja de estilo externa?

1. Crea un archivo con extensión ‘.css‘, por ejemplo: ‘estilos.css‘. 2. Escribe las reglas CSS en ese archivo:

```
body {
    background-color: #f9f9f9;
    color: #222;
}
h1 {
    color: #0056b3;
    text-align: center;
}
.boton {
    background: #007bff;
    color: white;
    border: none;
    border-radius: 4px;
    padding: 10px 20px;
}
```

¿Cómo vincular la hoja de estilo externa al HTML?

Coloca la siguiente línea dentro de la etiqueta `<head>` de tu documento HTML:

```
<head>
    ...
    <link rel="stylesheet" href="estilos.css">
    ...
</head>
```

```
<link rel="stylesheet" href="estilos.css">
```

Notas importantes:

- Puedes vincular varias hojas externas (se aplican en orden de aparición).
- El navegador descarga y aplica el CSS automáticamente.
- Si usas rutas relativas, asegúrate de que el archivo CSS esté en la ubicación correcta respecto al HTML.

Ventajas del CSS externo

- Permite reutilizar estilos en muchas páginas.
- Facilita el trabajo en equipo (diseñadores y desarrolladores pueden editar CSS sin tocar el HTML).
- Mejora la velocidad de carga (el CSS se cachea en el navegador).
- Hace el código más limpio y profesional.

Buenas prácticas

- Usa nombres descriptivos para los archivos CSS (‘main.css‘, ‘colores.css‘, etc.).
- Agrupa reglas relacionadas y comenta las secciones del CSS.
- Mantén la coherencia en la indentación y el formato.
- No dupliques reglas en diferentes archivos.
- Si tienes muchos estilos, divide en varios CSS por funcionalidad (por ejemplo: ‘layout.css‘, ‘typography.css‘).

Ejemplo:

```
<!-- index.html -->
<html>
<head>
    <meta charset="UTF-8">
    <title>Ejemplo CSS externo</title>
    <link rel="stylesheet" href="estilos.css">
</head>
<body>
    <h1>Bienvenido</h1>
    <button class="boton">Haz clic</button>
</body>
</html>

/* estilos.css */
h1 {
    color: #0056b3;
    font-family: Arial, sans-serif;
}
.boton {
    background: #007bff;
    color: white;
    border: none;
    border-radius: 4px;
    padding: 10px 20px;
    cursor: pointer;
}
.boton:hover {
    background: #0056b3;
}
```

La cascada y la herencia en CSS

La cascada y la herencia son dos conceptos fundamentales en CSS que determinan cómo se aplican los estilos a los elementos de una página web.

Cascada

La cascada se refiere al orden en que se aplican las reglas CSS. Cuando hay múltiples reglas que podrían aplicarse a un elemento, el navegador utiliza un conjunto de reglas para determinar cuál se aplica. Este orden se basa en tres factores:

- **Especificidad:** Las reglas más específicas tienen prioridad sobre las menos específicas. Por ejemplo, un selector de ID (#miElemento) es más específico que un selector de clase (.miClase).
- **Orden de aparición:** Si dos reglas tienen la misma especificidad, se aplica la que aparece más tarde en el CSS.
- **Importancia:** Las reglas marcadas como `!important` tienen la máxima prioridad, pero su uso debe ser evitado en la medida de lo posible.

Los navegadores otorgan un peso distinto a las hojas de estilo que, ordenadas de menor a mayor peso, son

- Estilos por defecto del navegador
- Hojas de estilo externas importadas (empleando el elemento @import dentro del elemento style en la cabecera del documento).
- Hojas de estilo internas (dentro de `<style>`).
- Estilos en línea (atributo `style` en el HTML).
- Declaraciones de estilo marcadas como !important.

Además de este orden, existe otra jerarquía de pesos que se aplican a las hojas de estilo creadas por la persona que ha diseñado la página web. Es importante entender esta jerarquía y tener en cuenta que las reglas de estilo que están al final de la lista sobreescribiran a las primeras. La siguiente lista, que como la anterior está ordenada de menor a mayor peso, muestra esta otra jerarquía:

- Hojas de estilo externas vinculadas (empleando el elemento link en la cabecera del documento).
- Hojas de estilo externas importadas (empleando el elemento @import dentro del elemento style en la cabecera del documento).
- Hojas de estilo incrustadas (empleando el elemento style en la cabecera del documento).
- Estilos en línea (empleando el atributo style en la etiqueta del elemento).
- Declaraciones de estilo marcadas como !important.

Herencia

La herencia permite que ciertos estilos se apliquen a los elementos secundarios de un elemento padre. Por ejemplo, si se establece un color de texto en un elemento `<body>`, todos los elementos de texto dentro de `<body>` heredarán ese color, a menos que se especifique lo contrario.

No todas las propiedades CSS son heredables. Las propiedades de texto, como `color` y `font-family`, son heredables, mientras que propiedades como `margin` y `padding` no lo son.

Resumen de las propiedades heredadas por defecto en HTML

color	text-align
font	visibility
letter-spacing	white-space
direction	border-collapse
word-spacing	quotes
line-height	list-style-image

4. Herramientas y test de verificación

Para garantizar la calidad y el correcto funcionamiento de las hojas de estilo, existen numerosas herramientas y tests de verificación. Estas permiten detectar errores, mejorar la accesibilidad, optimizar el rendimiento y asegurar el cumplimiento de estándares.

Validadores de CSS

- **W3C CSS Validator**: Herramienta oficial para comprobar la sintaxis y compatibilidad de las hojas de estilo. Disponible en: https://jigsaw.w3.org/css-validator/
- **CSSLint**: Analiza el código CSS y sugiere mejoras de calidad y buenas prácticas. https://csslint.net/

Herramientas de inspección y depuración

- **DevTools de navegador**: Todos los navegadores modernos (Chrome, Firefox, Edge, Safari) incluyen herramientas para inspeccionar, modificar y depurar CSS en tiempo real.
- **Extensiones útiles**: Ejemplo: "Web Developer" (disponible para Chrome y Firefox) añade utilidades para analizar estilos, estructuras y accesibilidad.

Test de compatibilidad y responsive

- **Responsinator**: Permite ver cómo se visualiza una web en diferentes dispositivos.
  https://www.responsinator.com/
- **BrowserStack / Sauce Labs**: Plataformas para probar el sitio en múltiples navegadores y sistemas operativos reales.

Accesibilidad y buenas prácticas

- **WAVE**: Analiza la accesibilidad de la web y detecta problemas visuales y estructurales.
  https://wave.webaim.org/
- **Lighthouse**: Herramienta de Google integrada en Chrome DevTools para auditar accesibilidad, rendimiento y buenas prácticas.

Recomendaciones

- Valida siempre tu CSS antes de publicar.
- Usa DevTools para depurar y experimentar con estilos.
- Comprueba la visualización en varios dispositivos y navegadores.
- Prioriza la accesibilidad y el rendimiento.
- Documenta los cambios y utiliza comentarios en el CSS.

Lee más:
HerramientasTestVerificacionEniun
InspeccionarElementosEniun

5. Tecnologías y frameworks

El desarrollo moderno de interfaces web se apoya en tecnologías y frameworks que facilitan la creación de estilos avanzados, responsivos y mantenibles. Estos recursos permiten ahorrar tiempo, aplicar buenas prácticas y lograr resultados profesionales.

5.1. Tecnologías de diseño

Media Queries

Las **Media Queries** son una característica fundamental de CSS que permiten adaptar el diseño de una web a diferentes dispositivos, tamaños de pantalla y condiciones del entorno. Son la base del diseño responsive, haciendo posible que una misma página se vea correctamente en móviles, tablets y ordenadores.

Sintaxis básica

Las Media Queries se definen usando la regla `@media` seguida de una condición. Dentro de la media query se colocan las reglas CSS que solo se aplicarán si se cumple la condición.

```
@media (condición) {
    /* Reglas CSS */
}
```

Ejemplo: Cambiar el color de fondo en pantallas pequeñas

```
body {
    background: #fff;
}

@media (max-width: 600px) {
    body {
        background: #e3f2fd;
    }
}
```

Condiciones más habituales

- `max-width`: Aplica estilos si el ancho de la pantalla es menor o igual al valor indicado.
- `min-width`: Aplica estilos si el ancho de la pantalla es mayor o igual al valor indicado.
- `orientation`: Permite detectar si el dispositivo está en modo vertical (`portrait`) u horizontal (`landscape`).
- `print`: Aplica estilos solo cuando se imprime la página.

Ejemplo: Diseño responsive de columnas

```
.contenedor {
    display: flex;
    flex-direction: row;
}
@media (max-width: 800px) {
    .contenedor {
        flex-direction: column;
    }
}
```

Buenas prácticas

- Utiliza `min-width` para el enfoque "mobile first".
- Agrupa las media queries al final del archivo CSS para facilitar el mantenimiento.
- Prueba el diseño en varios dispositivos y resoluciones.
- Usa unidades relativas (`em`, `rem`, `%`) para mayor flexibilidad.

Lee más:
Media Queries en Eniun
Guía sobre Media Queries en CSS
W3Schools
MDN Web Docs

Flexbox

Flexbox (Flexible Box Layout) es un módulo de CSS que facilita la distribución de espacio y la alineación de elementos en un contenedor, incluso cuando su tamaño es desconocido o dinámico.

Ejemplo básico:

```
<style>
.contenedor {
    display: flex;
    gap: 10px;
}
.item {
    background: #e3e3e3;
    padding: 20px;
    flex: 1;
}
</style>
<div class="contenedor">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
</div>
```

Permite alineación horizontal, vertical, ordenación y distribución de espacio de forma sencilla. Ideal para menús, barras de navegación y layouts simples.

Aprende
Flexbox en Eniun
Guía de introducción a Flexbox
W3Schools
MDN Web Docs

Juega
Flexbox Froggy

Grid Layout

Grid Layout es un sistema de maquetación bidimensional que permite crear diseños complejos de filas y columnas de forma sencilla y controlada.

Ejemplo básico:

```
<style>
.grid {
    display: grid;
    grid-template-columns: 1fr 2fr 1fr;
```

```
    gap: 10px;
}
.celda {
    background: #cce5ff;
    padding: 20px;
}
</style>
<div class="grid">
    <div class="celda">A</div>
    <div class="celda">B</div>
    <div class="celda">C</div>
</div>
```

Permite crear layouts responsivos, dashboards y estructuras complejas sin necesidad de floats ni hacks. Grid y Flexbox pueden combinarse para lograr resultados avanzados.
Aprende
Grid Layout en Eniun
Guía de introducción a Grid Layout
W3Schools
MDN Web Docs
Juega
CSS Grid Garden

5.2. Frameworks CSS más populares

- **Bootstrap**: El framework CSS más usado. Proporciona un sistema de rejilla (grid), componentes listos para usar (botones, menús, formularios) y utilidades para diseño responsive.
  Aprende
  https://www.eniun.com/tutorial-bootstrap-5/
  https://getbootstrap.com/docs/5.3/getting-started/introduction/
- **Materialize**: Basado en las directrices de diseño de Google Material Design. Ofrece componentes modernos y animaciones.
  https://materializecss.com/
- **Tailwind CSS**: Framework de utilidades. Permite aplicar estilos directamente en las clases del HTML, favoreciendo la personalización y la rapidez.
  https://tailwindcss.com/
- **Bulma**: Basado en Flexbox, sencillo y modular.
  https://bulma.io/
- **Foundation**: Alternativa avanzada a Bootstrap, con herramientas para accesibilidad y prototipado rápido.
  https://get.foundation/

Ventajas de usar frameworks

- Aceleran el desarrollo y reducen errores.

- Garantizan compatibilidad entre navegadores.
- Incluyen componentes accesibles y responsivos.
- Fomentan la coherencia visual en todo el proyecto.
- Permiten personalización mediante variables y temas.

Otras tecnologías y utilidades

- **Normalize.css**: Restablece los estilos por defecto de los navegadores para evitar inconsistencias.
  https://necolas.github.io/normalize.css/
- **Animate.css**: Biblioteca de animaciones CSS listas para usar.
  https://animate.style/
- **Iconos**: Font Awesome, Material Icons, Bootstrap Icons, etc.

Ejemplo de uso: Bootstrap

```
<!-- Enlazar Bootstrap desde CDN -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/boo

<!-- Botón con clases Bootstrap -->
<button class="btn btn-primary">Guardar</button>
```

Recomendaciones

- Lee la documentación oficial de cada framework antes de usarlo.
- Personaliza solo lo necesario para no perder la coherencia.
- No abuses de los frameworks: aprende CSS "puro" para entender lo que ocurre detrás.
- Mantén los frameworks actualizados para evitar vulnerabilidades.

6. Preprocesadores de estilos. Variables, mixins y funciones

Los preprocesadores de estilos son herramientas que amplían las capacidades de CSS, permitiendo usar variables, funciones, anidamiento, mixins y otras utilidades avanzadas. El código se escribe en un lenguaje especial (como SASS, SCSS o LESS) y luego se compila a CSS estándar.

Ventajas de los preprocesadores

- Permiten reutilizar código y evitar repeticiones.
- Facilitan la organización y el mantenimiento de proyectos grandes.
- Añaden lógica (condicionales, bucles) y estructuras avanzadas.
- Mejoran la legibilidad y escalabilidad del CSS.

Principales preprocesadores

- **SASS/SCSS**: El más popular, con sintaxis flexible y gran comunidad.
  https://sass-lang.com/
- **LESS**: Muy usado en proyectos legacy y compatible con variables y mixins.
  https://lesscss.org/
- **Stylus**: Sintaxis muy concisa y flexible.
  https://stylus-lang.com/

Variables

Permiten definir valores reutilizables para colores, tamaños, fuentes, etc.

```
$color-principal: #007bff;
body {
    color: $color-principal;
}
```

Mixins

Son bloques de código reutilizables que se pueden incluir en varias reglas.

```
@mixin redondeado($radio) {
    border-radius: $radio;
    box-shadow: 0 2px 4px #ccc;
}
.boton {
    @include redondeado(8px);
}
```

Funciones

Permiten realizar operaciones y devolver valores, por ejemplo para manipular colores.

```
.alerta {
    background: lighten($color-principal, 20%);
}
```

Ejemplo completo (SCSS)

```
$color-fondo: #f8f9fa;
$color-boton: #007bff;

@mixin sombra {
    box-shadow: 0 2px 8px #bbb;
}

body {
    background: $color-fondo;
}
.boton {
    background: $color-boton;
    color: white;
    @include sombra;
    border-radius: 6px;
}
```

¿Cómo se usan en un proyecto?

- Escribe el código en archivos ‘.scss‘, ‘.sass‘ o ‘.less‘.
- Usa una herramienta (Node.js, extensiones de editor, scripts) para compilar a CSS estándar.
- Enlaza el CSS generado en tu HTML como siempre.

Recomendación: Aunque los preprocesadores son muy potentes, es importante conocer bien CSS puro antes de usarlos en profundidad.

Tutorial de Sass en Eniun: https://www.eniun.com/tutorial-sass/